

HiCAMS 선박 CCTV 관제

AWS IoT 기반 To-Be 아키텍처 제안

고객: KSOE

현행 K3s 에지 클러스터를 AWS IoT 서비스와 연동하는 두 가지 아키텍처 옵션 제안 및 권장 전환 경로 제시

프로젝트 배경 및 Phase 계획

Phase 1

OTA 업데이트

원격 소프트웨어 배포 자동화

- Greengrass 컴포넌트 기반 배포
- 멀티 선박 동시 배포 지원
- 롤백 및 버전 관리 지원

AWS IoT Greengrass

Phase 2

비디오 스트리밍 + 에지 추론 및 이상감지 알림

YOLO실시간 분석

- 에지-클라우드 하이브리드 처리
- 실시간 에지 추론 결과 동기화
- 비디오 클립 전송

AWS IoT Greengrass + Amazon S3

Phase 3

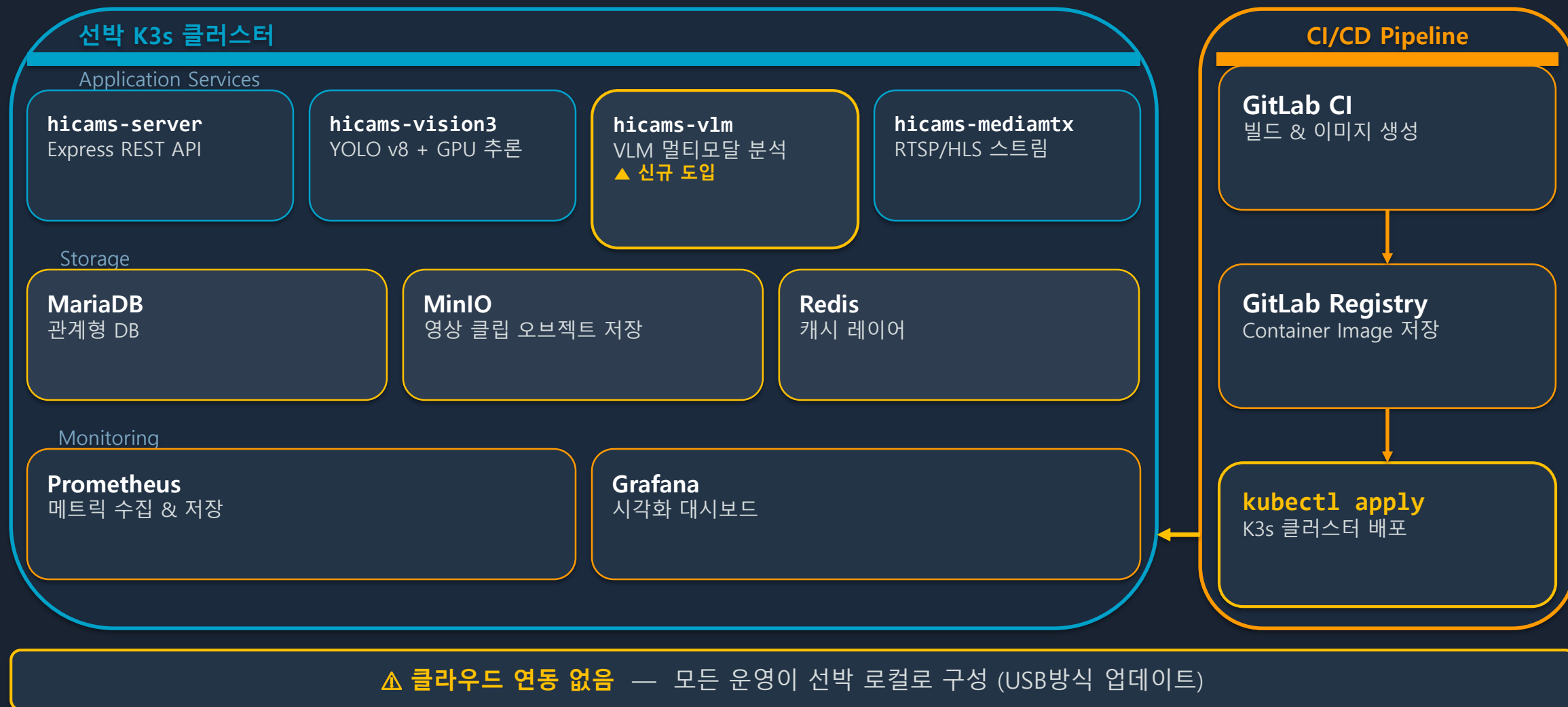
선주 모니터링 서비스

대시보드, 알림, 데이터 분석

- 통합 관제 대시보드 제공
- 이상 감지 알림 자동화
- 선박 운항 데이터 분석

AWS 클라우드 통합 플랫폼

AS-IS 현행 시스템 구성



네트워크 제약 및 클라우드 연결 방식

⚠ **방화벽 제약 조건** 선박 방화벽 하위 환경 — 외부 인바운드 직접 접근 불가

허용 아웃바운드 포트: **443 (HTTPS)** · **8883 (MQTT over TLS)**

해결책 1

AWS IoT Secure Tunneling

:443 (HTTPS) 아웃바운드 개시

- 에이전트 아웃바운드 연결 → 양방향 보안 터널 생성
- 클라우드 → 선박 원격 접근 가능 (OTA, 트러블슈팅)
- **방화벽 정책 변경 불필요**

해결책 2

MQTT over TLS

:8883 아웃바운드 → IoT Core 상시 연결

- 선박이 IoT Core에 지속적인 연결 유지
- 이벤트 및 텔레메트리 데이터 실시간 전송
- **방화벽 정책 변경 없이 아웃바운드 개시**

선박 에이전트

아웃바운드 :443 개시

AWS IoT Secure Tunneling

양방향 보안 터널

OTA 명령 · 원격 관리

클라우드 → 선박 원격 접근

선박 서비스

아웃바운드 :8883 개시

AWS IoT Core

MQTT over TLS 상시 연결

이벤트 · 텔레메트리

실시간 전송

핵심 설계 원칙 — Greengrass 배치 전략

📌 설치 위치

Greengrass는 K3s 클러스터 외부,
호스트 OS의 systemd 서비스로 설치
AWS 레퍼런스 아키텍처 표준 패턴 준수

⚠️ 부트스트랩 데드락 방지

OTA 에이전트(Greengrass)가 K3s 내부에 있으면 K3s 장애 시 복구 불가
→ 부트스트랩 데드락, 자가 복구 불가

🔄 통신 경로

Greengrass → K3s API Server (localhost:6443) via kubeconfig
K3s NodePort → Greengrass IPC 노출
→ 자가 복구 OTA의 전제 조건

Host OS (선박 컴퓨터)

Greengrass Core

systemd 서비스

↑ AWS IoT Core

K3s Cluster

K3s API Server

localhost:6443

애플리케이션 파드

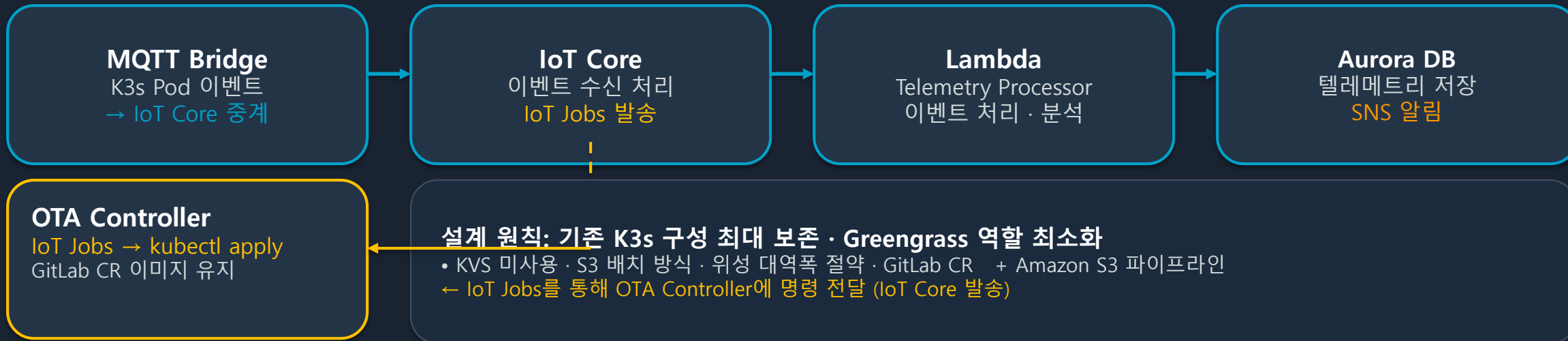
AI 추론 / 영상 처리

NodePort 서비스

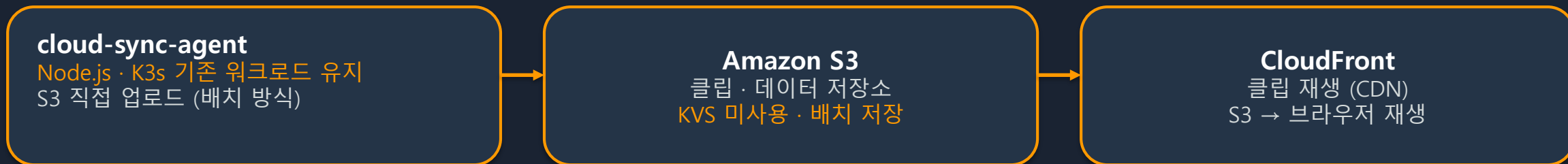
Greengrass IPC 노출

Phase 1 – OTA 기능 검증

▶ **Greengrass 이벤트 흐름** (Components 2개: MQTT Bridge · OTA Controller)



▶ **K3s 기존 데이터 흐름** (기존 코드 변경 없음 · S3 배치 방식)



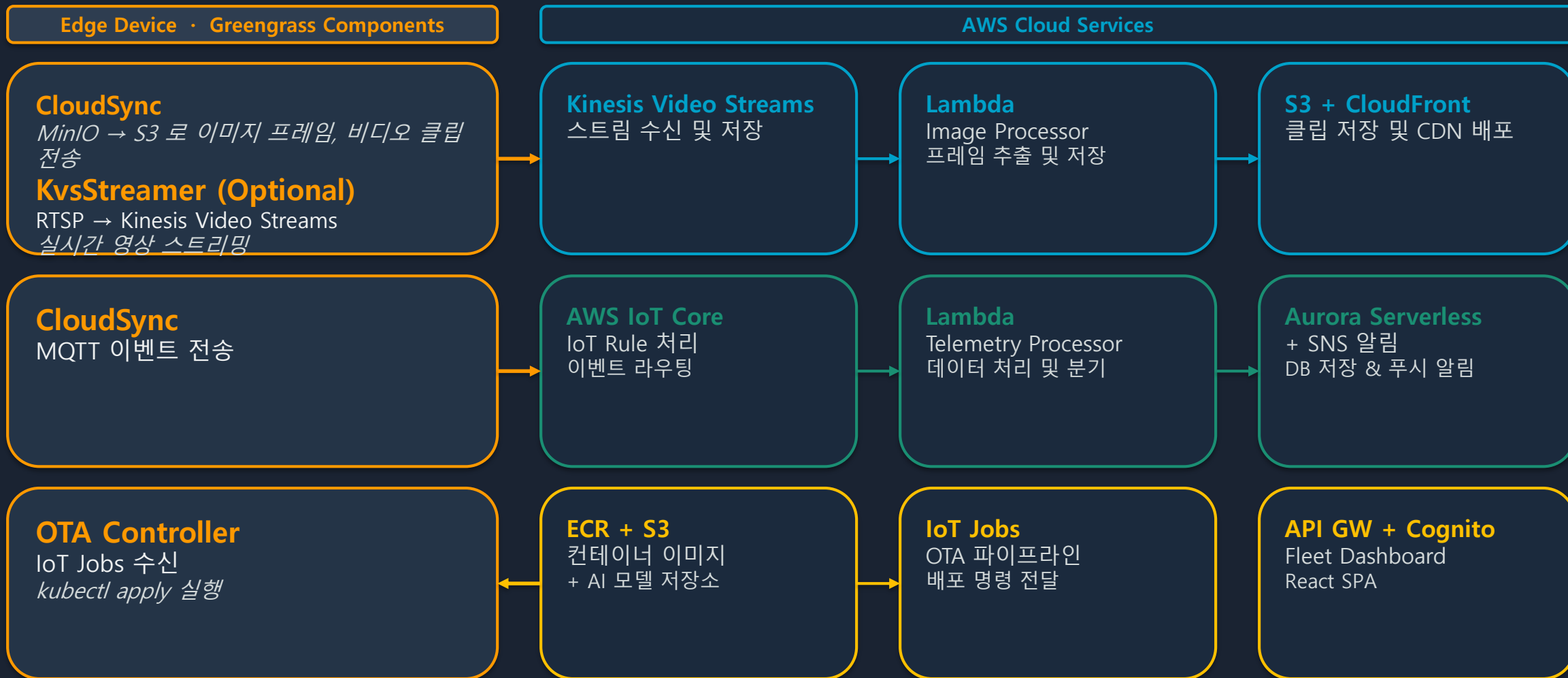
▶ **기존 코드 변경 최소**

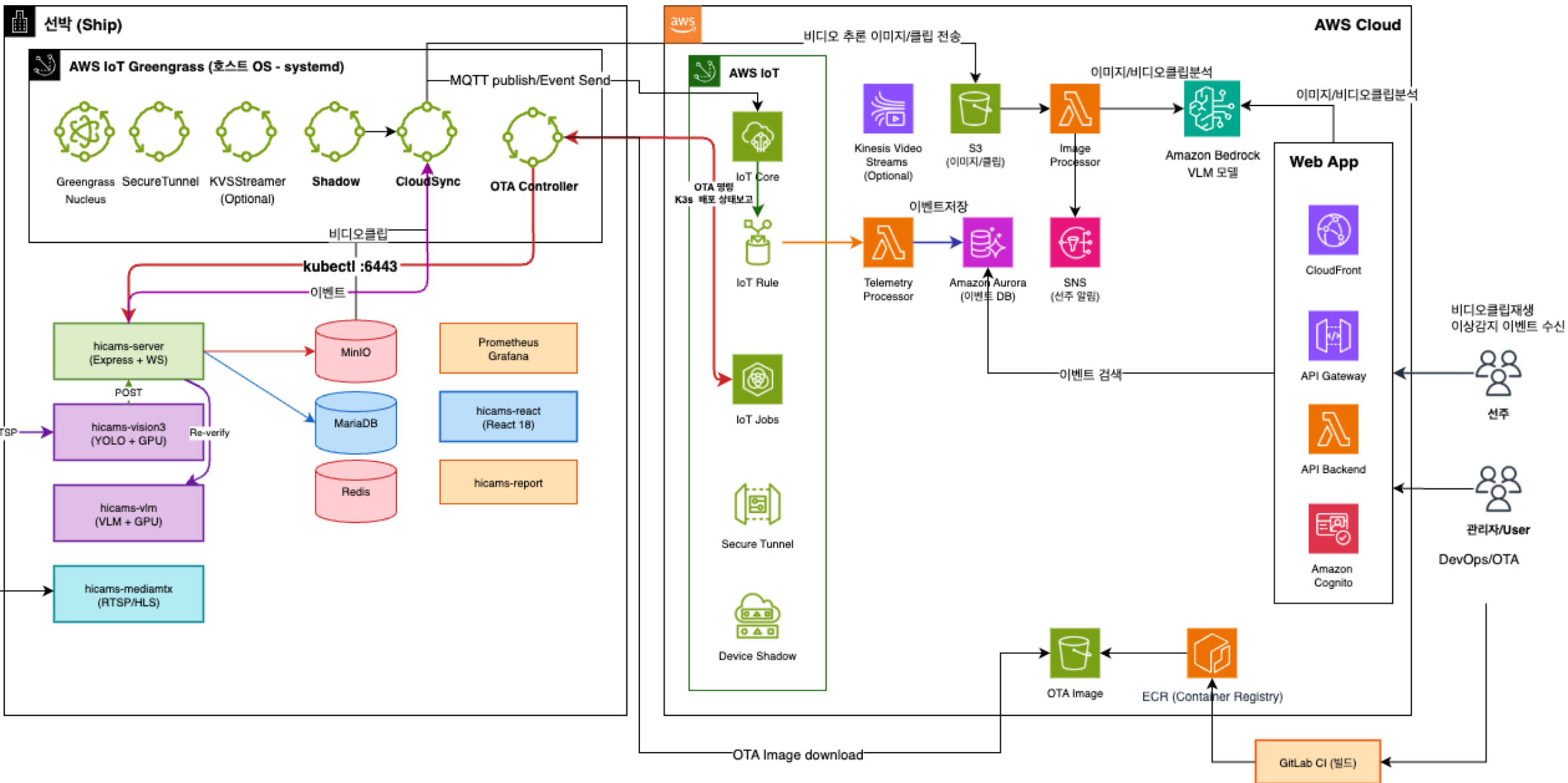
리팩토링 없이 Phase 1 즉시 착수 가능

▶ **위성 대역폭 절약**

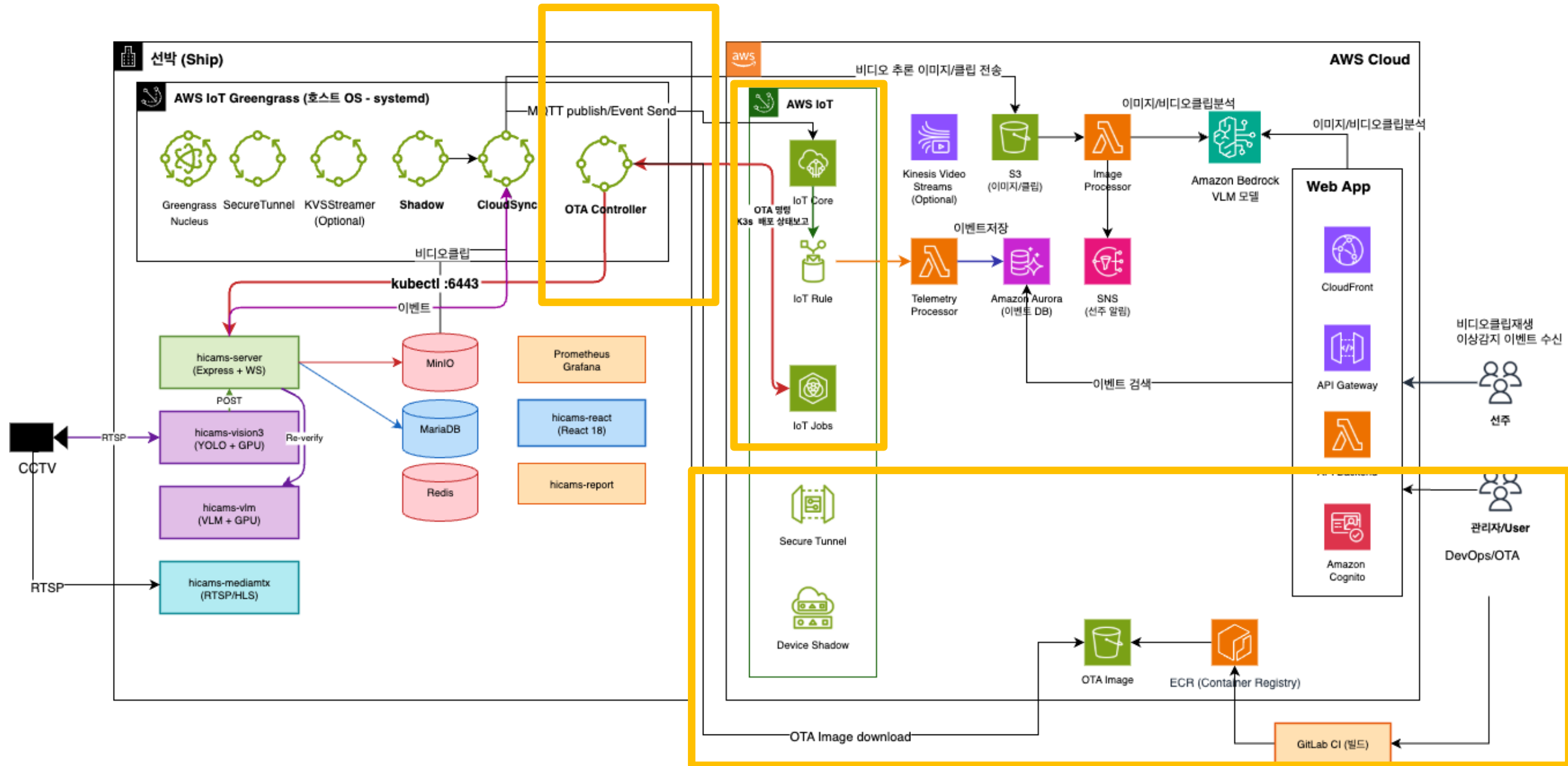
S3 배치 방식 · KVS 스트리밍 없음

Phase2: 비디오 스트리밍 + 에지 추론 및 이상감지 알림

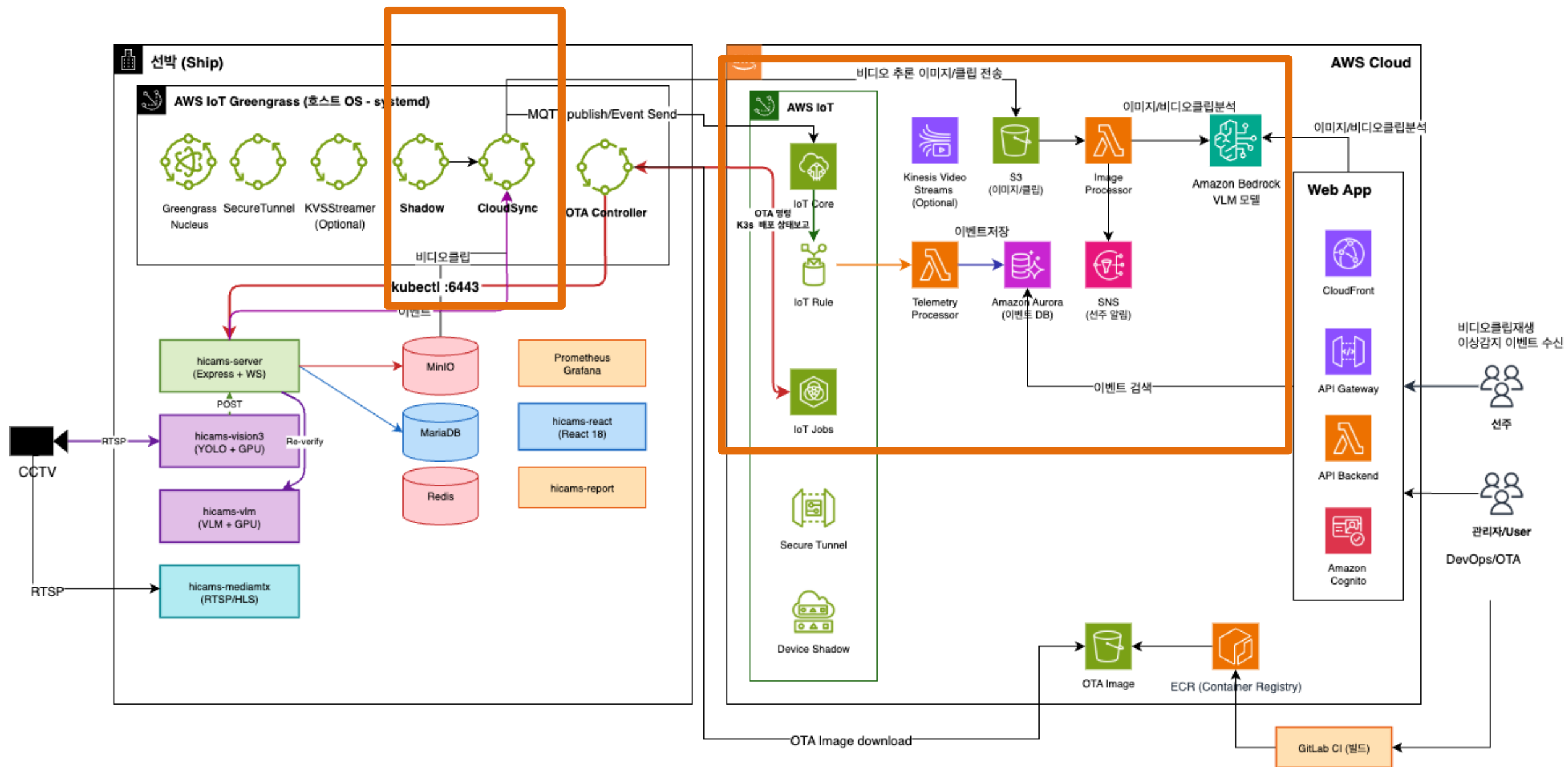




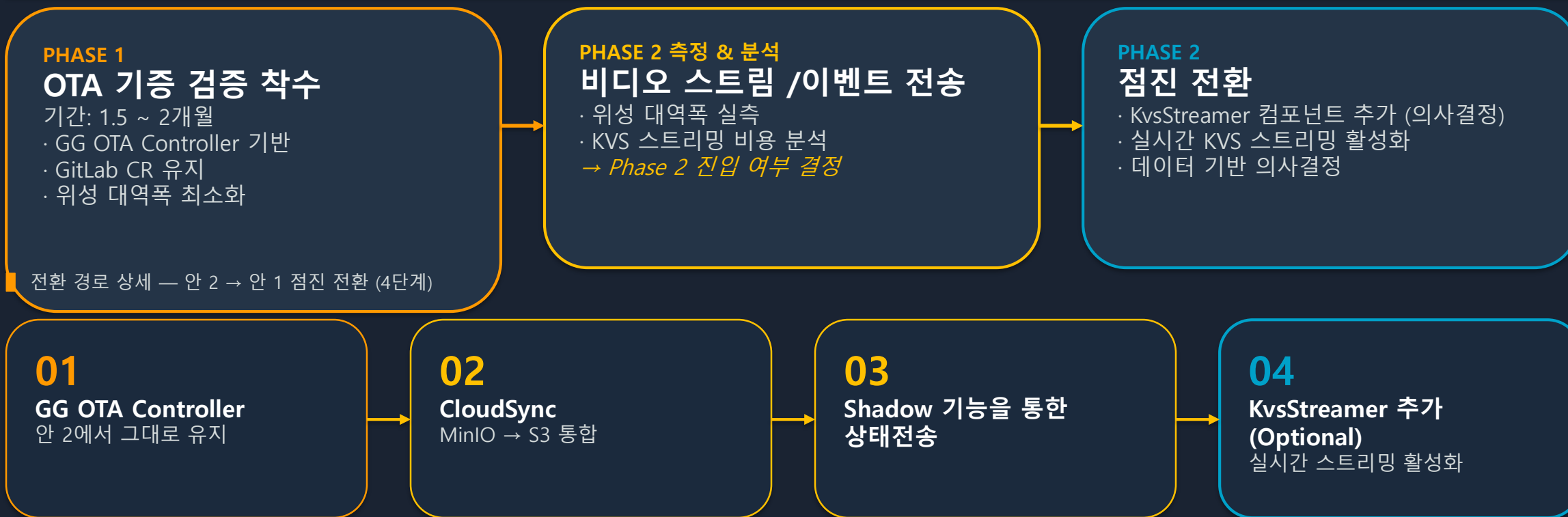
Phase1



Phase2



권장 전략 — Phase별 점진적 전환 경로

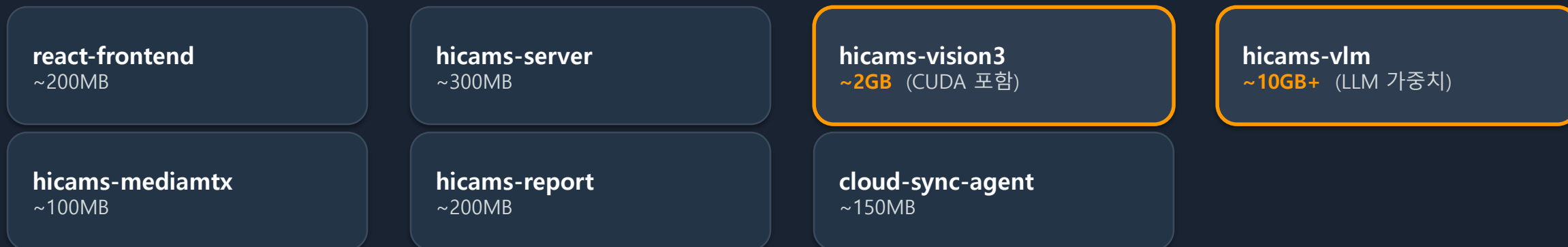


전환 경로 상세 — 안 2 → 안 1 점진 전환 (4단계)

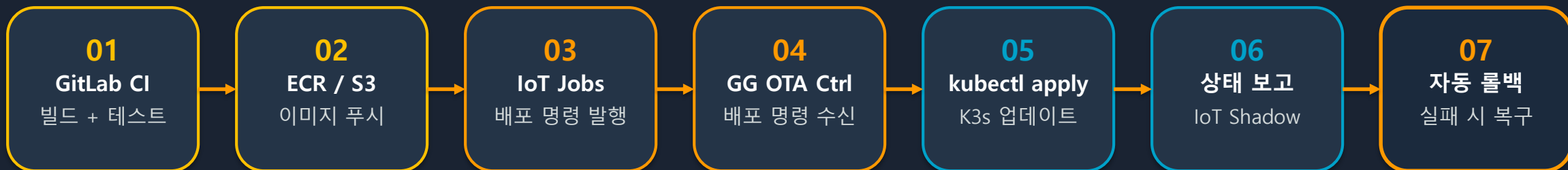
⚠ **리스크 최소화:** 각 Phase는 독립적으로 검증 후 다음 단계 진행. 측정 결과에 따라 안 1 전환 여부를 결정합니다.

OTA 파이프라인 — 대상 컴포넌트 및 배포 흐름

OTA 대상 컴포넌트 (7개)



배포 파이프라인 흐름



※ 대용량 컴포넌트 **hicams-vlm (~10GB+)**, **hicams-vision3 (~2GB)**는 Pre-staging(사전 캐싱) 전략 필요 | 07 자동 롤백은 배포 실패 시에만 트리거(점선 화살표)
배에서 OTA가 가능한 상황인지 알림을 줄 수 있는지 필요

열악한 네트워크 환경 대응 — OTA 안정성 설계

선박 위성 통신 환경

대역폭 수백 Kbps ~ 수 Mbps

지연 600ms+ (고지연)

가용성 간헐적 단절 발생

변동성 항해 중 대역폭 급변

OTA 안정성 설계가 필수적인 환경

안정성 원칙

- ① 오프라인 큐잉
- ② 이미지 사전 캐싱
- ③ Docker File 사이즈 최소화

01 오프라인 큐잉

네트워크 단절 시 IoT Jobs를 Greengrass 로컬 큐에 보존 → 연결 복구 시 자동 재개 (K3s 재시작과 무관)

02 이미지 사전 캐싱 (Pre-staging)

hicams-vision3(~2GB) 등 대용량 이미지를 로컬 MinIO에 미리 다운로드 → 다운로드와 배포 분리, 실패가 서비스에 무영향

03 Docker File 사이즈 최소화

변경된 Docker 레이어만 전송 (Docker Layer Cache) — 200MB 이미지 중 변경분 20MB만 전송으로 대역폭 절약 - 최적화 테스트 필요

04 단계적 배포

테스트 선박 1척 → 그룹(5척) → 전체 선단 순차 배포, 각 단계 24시간 모니터링 후 다음 단계 진행

OTA 시나리오별 동작 흐름

시나리오 1 — 정상 배포

- ① IoT Jobs 명령 수신
- ② 이미지 Pull (ECR / S3)
- ③ kubectl apply 실행
- ④ rollout status 확인 → OK
- ⑤ **Shadow: v2.1.0 | status: SUCCESS**

시나리오 2 — 네트워크 단절

- ① 이미지 Pull 시작
- ② *위성 연결 끊김 (수 시간)*
- ③ Greengrass 큐에 Job 보존
- ⑤ **배포 완료**

시나리오 3 — 선박 정박 시 업데이트

- ① 주기적인 대역폭 체크
- ② *선박 정박 시 job queue 체크*
- ③ IoT Jobs의 명령 수신하여 이미지 pull
- ④ kubectl 명령 실행후 결과에 대한 상태 shadow에 공유
- ⑤ **Shadow: ROLLED_BACK | SNS 알림 발송**

시나리오 4 — 장기 오프라인 복구

- ① 선박 수일간 오프라인 상태
- ② 클라우드 IoT Jobs 큐에 다수 누적
- ③ 연결 복구
- ④ **최신 버전만 적용 (중간 버전 스킵)**
→ *대역폭 절약 효과*

다음 단계

즉시 확인 필요

- 방화벽 포트 443/8883 아웃바운드 정책
- 실제 위성 대역폭 측정값
- K3s 노드 사양 – VLM 탑재 가능한 사양인지
- 운항 중 카메라 수량 및 RTSP 스트림 수

아키텍처 결정

- Greengrass 도입 범위 결정
- 컨테이너 레지스트리 선택 (ECR vs GitLab CR)
- 비디오 전송 방식 (KVS vs S3 배치)
- 데이터 동기화 방식 결정
- Local VLM 필요성 여부 (클라우드 멀티모달 방식을 통해 비디오클립 분석)

▷ 다음 단계

01

아키텍처 확정 워크샵

2주 내 워크숍 일정 조율

아키텍처 핵심 결정 사항 확정

파트너사와 아키텍처 협의 R&R 정리

02

IoT Secure Tunneling PoC

AWS IoT Secure Tunneling 연결성

보안 채널 통합 검증

03

Greengrass OTA Controller 프로토타입

컨테이너 배포 자동화 구현

프로토타입 개발 착수

Greengrass vs K3s — 배포 안정성 레이어 비교

Greengrass Nucleus

OTA 컴포넌트 배포 최적화

✓ S3 자동 재시도

지수 백오프 내장 · 중단 지점 재시작

✓ SHA-256 무결성 검증

다운로드 완료 후 자동 체크

✓ 자동 롤백

배포 실패 시 이전 버전 자동 복원

✓ 오프라인 큐잉

위성 단절 → 재연결 시 자동 재개

✓ GG 컴포넌트 배포에 최적

VS

K3s containerd

대용량 이미지 Pull — 위성 환경 부적합

△ 레이어 단위 재시도

레이어 단위 기본 재시도 기능 존재

✗ 레이어 내 이어받기(Resume) 미지원

중단 시 해당 레이어 처음부터 재시작

✗ VLM 단일 레이어 7GB+ 위험

연결 끊김 → 전체 레이어 재다운로드

✗ 위성 환경 배포 불가 리스크

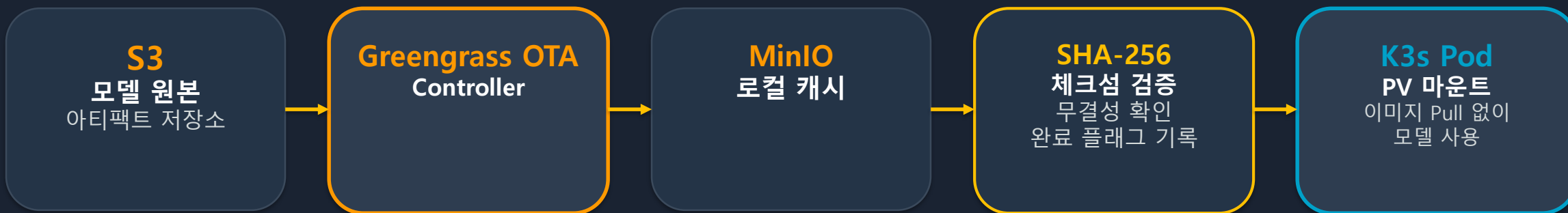
실질적 대용량 직접 Pull 사용 불가

✗ 대용량 모델 직접 Pull 사용 금지

선박 위성 환경 — 사전 캐싱(Pre-staging) 패턴

핵심 원칙: 대용량 모델은 컨테이너 이미지에서 분리 — 이미지 = 코드만(수백MB 경량) / 모델 = PV 마운트

▶ 전송 흐름



▶ 추가 안전장치 (3가지)

01

대역폭 스로틀링

CCTV 스트리밍 우선 대역폭 확보
OTA 전송 상한 설정으로 운영 보호

02

야간 스케줄링

대역폭 여유 시간대에만
대용량 모델 다운로드 예약 실행

03

Cloudfront/S3 Transfer Acceleration

위성 고지연 TCP 느린 시작 완화
엣지 PoP 경유로 레이턴시 단축

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/transfer-acceleration-examples.html>

감사합니다

Q&A 세션 — 질문을 받겠습니다